Criteria	Black Box Testing	White Box Testing
Definition	Black Box Testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is NOT known to the tester	White Box Testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is known to the tester.
Levels Applicable To	Mainly applicable to higher levels of testing: Acceptance Testing System Testing	Mainly applicable to lower levels of testing: Unit Testing Integration Testing
Responsibility	Generally, independent Software Testers	Generally, Software Developers
Programming Knowledge	Not Required	Required
Implementation Knowledge	Not Required	Required
Basis for Test Cases	Requirement Specifications	Detail Design

VIII. Debugging

1. Concept and need of Debugging

(Question: Describe the concept and need of debugging in software testing- 6 Marks)

i. Debugging is the process of locating and fixing or bypassing bugs (errors) in computer program code or the engineering of a hardware device.

ii. To *debug* a program or hardware device is to start with a problem, isolate the source of the problem, and then fix it.
iii. A user of a program that does not know how to fix the problem may learn enough about the problem to be able to avoid it until it is permanently fixed.
iv. When someone says they've debugged a program or "worked the bugs out" of a program, they imply that they fixed it so that the bugs no longer exist.
v. Debugging is a necessary process in almost any new software or hardware development process, whether a commercial product or an enterprise or personal application program.
vi. For complex products, debugging is done as the result of the unit test for the smallest unit of a system, again at component test when parts are brought together, again at system test when the product is used with other existing products, and again during customer beta test, when users try the product out in a real world situation.
vii. Because most computer programs and many programmed hardware devices contain thousands of lines of code, almost any new product is likely to contain a few bugs.
viii. Invariably, the bugs in the functions that get most use are found and fixed first. An early version of a program that has lots of bugs is referred to as "buggy."
ix. Debugging tools (called debuggers) help identify coding errors at various development stages. Some programming language packages include a facility for checking the code for errors as it is being written.

2. Characteristics of bugs

(Question: Explain the various characteristics of bugs – 4 Marks)

i. A defect is an error or a bug, in the application which is created.
ii. A programmer while designing and building the software can make mistakes or error.
iii. These mistakes or errors mean that there are flaws in the software. These are called defects.
iv. When actual result deviates from the expected result while testing a software application or product then it results into a defect.

v. Hence, any deviation from the specification mentioned in the product functional specification document is a defect. In different organizations it's called differently like bug, issue, incidents or problem.
vi. When the result of the software application or product does not meet with the end user expectations or the software requirements then it results into a Bug or Defect.
vii. These defects or bugs occur because of an error in logic or in coding which results into the failure or unpredicted or unanticipated results.

IX. Debugging Strategies

(Question: Explain any two debugging strategies- 8 marks)

1. Brute Force

i. This method is most common and least efficient for isolating the cause of a software error.
ii. We apply this method when all else fail. In this method, a printout of all registers and relevant memory locations is obtained and studied.
iii. All dumps should be well documented and retained for possible use on subsequent problems.

2. Back Tracking

i. It is a quite popular approach of debugging which is used effectively in case of small applications.
ii. The process starts from the site where a particular symptom gets detected, from there on backward tracing is done across the entire source code till we are able to lay our hands on the site being the cause.
iii. As the number of source lines increases, the number of potential backward paths may become unmanageably large.

3. Cause Elimination(Induction and Deduction)

i. The third approach to debugging, because elimination, is manifested by induction or deduction and introduces the concept of binary partitioning. This approach is also called induction and deduction.
ii. Data related to the error occurrence are organized to isolate potential causes.
iii. A "cause hypothesis" is devised and the data are used to prove or disprove the hypothesis.
iv. A list of all possible causes is developed and tests are conducted to eliminate each.

v. If initial tests indicate that a particular cause hypothesis shows promise, the data are refined in an attempt to isolate the bug.